INFORME

I. PORTADA

Tema: **Árbol Binario de Búsqueda para organigrama empresarial**
Unidad de Organización Curricular: BÁSICA
Nivel y Paralelo: 3B
Alumnos participantes: Amaguaña Shirley
Asignatura: Estructura de Datos
Docente: Ing. José Caiza

II. INFORME

2.1 Objetivo

Implementar en C++ un Árbol Binario de Búsqueda (BST) para organizar empleados de una empresa usando un código numérico como clave. El ejercicio representa un organigrama empresarial donde se pueden identificar claramente la raíz, los niveles, los nodos internos y las hojas.

2.2 Entregables:

Link del repositorio GitHub.
Código fuente en C++.
README completo.
Capturas de ejecución.
Explicación breve de raíz, nodo interno, hoja, nivel y altura.
Participación de cada integrante mediante commits.

3. Resultados obtenidos

Conceptos clave

¿Qué es un Árbol Binario de Búsqueda?

Un Árbol Binario de Búsqueda (BST, por sus siglas en inglés Binary Search Tree) es una estructura de datos jerárquica en la que cada nodo puede tener como máximo dos hijos: uno izquierdo y uno derecho. La regla fundamental que define a un BST es:

- Todo nodo del subárbol izquierdo tiene un valor menor que el nodo padre.
- Todo nodo del subárbol derecho tiene un valor mayor que el nodo padre.
- No se permiten valores duplicados.

Gracias a esta regla, las operaciones de búsqueda, inserción y eliminación tienen una complejidad promedio de O(log n), lo que las hace mucho más eficientes que una lista lineal.

Concepto	Definición
Raíz	El nodo principal del árbol. Es el primer elemento insertado. En nuestro árbol: código 50 (Empresa UTA).
Nodo interno	Nodo que tiene al menos un hijo. Ejemplos: código 30 (Gerente Ventas) y código 70 (Gerente Finanzas).
Hoja	Nodo que no tiene ningún hijo. Es el nivel más bajo. Ejemplos: códigos 20, 40, 60 y 80.
Altura	Número de niveles del árbol desde la raíz hasta la hoja más profunda. En nuestro árbol: 3 niveles.
Subárbol izquierdo	Todos los nodos con código menor al nodo padre.
Subárbol derecho	Todos los nodos con código mayor al nodo padre.

Concepto	Definición
Puntero nullptr	En C++, indica que un nodo no tiene hijo en esa dirección (está vacío).

Los tres recorridos del árbol
Un recorrido es el orden en que se visitan todos los nodos del árbol. Existen tres tipos:

Recorrido	Orden de visita	Resultado con datos de prueba	Para qué sirve
Inorden	Izq → Raíz → Der	20, 30, 40, 50, 60, 70, 80	Lista ordenada de menor a mayor
Preorden	Raíz → Izq → Der	50, 30, 20, 40, 70, 60, 80	Ver jerarquía, copiar el árbol
Postorden	Izq → Der → Raíz	20, 40, 30, 60, 80, 70, 50	Procesar hijos antes que padres

4. Código Fuente
4.1 Estructura general del programa
El programa está organizado en tres bloques principales:
- struct Empleado — define los datos de cada empleado (código, nombre, cargo).
- struct Nodo — define cada cajita del árbol con su empleado y dos punteros.
- class ArbolBST — contiene todas las operaciones del árbol con menú interactivo.

4.2 struct Empleado
Un struct es una plantilla que agrupa varios datos relacionados. Aquí definimos qué información tiene cada empleado:

```
//  STRUCT EMPLEADO
//  Representa los datos de cada persona en el organigrama.

struct Empleado {
    int    codigo;   // Clave del BST: decide a donde va en el arbol
    string nombre;   // Nombre del empleado
    string cargo;    // Cargo dentro de la empresa
};
```

4.3 struct Nodo
Cada nodo es una cajita que guarda un Empleado y dos punteros: uno al hijo izquierdo y otro al hijo derecho.

```
//  STRUCT NODO
//  Es cada "cajita" del arbol. Guarda un Empleado y dos
//  punteros: uno al hijo izquierdo y otro al hijo derecho.
struct Nodo {
    Empleado dato;       // Los datos del empleado
    Nodo*    izquierdo;  // Puntero al hijo izquierdo
    Nodo*    derecho;    // Puntero al hijo derecho

    // Constructor: cuando se crea un Nodo, sus hijos empiezan vacios
    Nodo(Empleado emp) {
        dato       = emp;
        izquierdo  = nullptr;
        derecho    = nullptr;
    }
};
```

4.4 Función insertar (recursiva)

Esta es la función más importante. Usa recursión para encontrar el lugar correcto donde colocar el nuevo empleado, siguiendo la regla del BST:

```
// INSERTAR (privado, recursivo)
//   - codigo menor  -> va al SUBÁRBOL IZQUIERDO
//   - codigo mayor  -> va al SUBÁRBOL DERECHO
//   - codigo igual  -> no se inserta (codigo ya existe)
Nodo* insertar(Nodo* nodo, Empleado emp) {
    // Caso base: llegamos a un lugar vacio -> creamos el nodo aqui
    if (nodo == nullptr) {
        return new Nodo(emp);
    }

    // Comparamos el codigo del nuevo empleado con el del nodo actual
    if (emp.codigo < nodo->dato.codigo) {
        // El codigo es menor: buscamos lugar en el lado izquierdo
        nodo->izquierdo = insertar(nodo->izquierdo, emp);
    } else if (emp.codigo > nodo->dato.codigo) {
        // El codigo es mayor: buscamos lugar en el lado derecho
        nodo->derecho = insertar(nodo->derecho, emp);
    } else {
        // El codigo ya existe en el arbol
        cout << "  [!] El codigo " << emp.codigo << " ya existe en el arb
    }

    return nodo;
```

La recursión funciona así: la función se llama a sí misma con el hijo izquierdo o derecho según la comparación, hasta encontrar un nullptr donde insertar el nuevo nodo.

4.5 Función buscar (recursiva)

Sigue la misma lógica que insertar, pero en lugar de crear un nodo, retorna el puntero al nodo encontrado (o nullptr si no existe):

```
// BUSCAR (privado, recursivo)
// Recorre el arbol comparando codigos hasta encontrar o
// llegar a un nullptr (no existe)
Nodo* buscar(Nodo* nodo, int codigo) {
    // Caso base: nodo vacio (no encontrado) o codigo coincide
    if (nodo == nullptr || nodo->dato.codigo == codigo) {
        return nodo;
    }

    // Decidimos en que direccion seguir buscando
    if (codigo < nodo->dato.codigo) {
        return buscar(nodo->izquierdo, codigo);
    } else {
        return buscar(nodo->derecho, codigo);
```

4.6 Función altura (recursiva)

Calcula el número de niveles del árbol tomando el camino más largo desde la raíz hasta una hoja:

```
// ALTURA (privado, recursivo)
// La altura es el numero de niveles del arbol.
// Cuenta el camino mas largo desde la raiz hasta una hoja.
// Formula: 1 + max(altura_izquierda, altura_derecha)
int altura(Nodo* nodo) {
    if (nodo == nullptr) {
        return 0;   // Nodo vacio tiene altura 0
    }
    int alturaIzq = altura(nodo->izquierdo);
    int alturaDer = altura(nodo->derecho);
    return 1 + max(alturaIzq, alturaDer);  // +1 por el nodo actual
}
```

4.7 Función mostrarHojas

Un nodo es hoja cuando sus dos punteros son nullptr (no tiene hijos). La función revisa esto en cada nodo:

```
// MOSTRAR HOJAS (privado, recursivo)
// Una hoja es un nodo que NO tiene hijos (ambos son nullptr).
// Los nodos hoja son los empleados del nivel mas bajo.
void mostrarHojas(Nodo* nodo) {
    if (nodo != nullptr) {
        // Si no tiene ni hijo izquierdo ni derecho: es hoja
        if (nodo->izquierdo == nullptr && nodo->derecho == nullptr) {
            mostrarEmpleado(nodo);
        }
        mostrarHojas(nodo->izquierdo);  // Revisamos la izquierda
        mostrarHojas(nodo->derecho);    // Revisamos la derecha
    }
}
```

4.8 Funciones de los recorridos

```
// RECORRIDO INORDEN: Izquierda -> Raiz -> Derecha
// Resultado: los empleados salen ORDENADOS de menor a mayor codigo
void inorden(Nodo* nodo) {
    if (nodo != nullptr) {
        inorden(nodo->izquierdo);   // 1ro visita la izquierda
        mostrarEmpleado(nodo);      // 2do muestra el actual
        inorden(nodo->derecho);     // 3ro visita la derecha
    }
}


// RECORRIDO PREORDEN: Raiz -> Izquierda -> Derecha
// Resultado: muestra la raiz PRIMERO, luego sus hijos
void preorden(Nodo* nodo) {
    if (nodo != nullptr) {
        mostrarEmpleado(nodo);      // 1ro muestra el actual
        preorden(nodo->izquierdo);  // 2do visita la izquierda
        preorden(nodo->derecho);    // 3ro visita la derecha
    }
}

// RECORRIDO POSTORDEN: Izquierda -> Derecha -> Raiz
// Resultado: los hijos salen ANTES que sus padres
// Util para eliminar el arbol (liberar memoria)
void postorden(Nodo* nodo) {
    if (nodo != nullptr) {
        postorden(nodo->izquierdo); // 1ro visita la izquierda
        postorden(nodo->derecho);   // 2do visita la derecha
        mostrarEmpleado(nodo);      // 3ro muestra el actual
    }
}
```

LINK DEL GITHUB: https://github.com/shirley6126/arbol-bst-empresa-cpp

5. Conclusión

Este ejercicio refuerza conceptos fundamentales de la programación en C++: el uso de struct y class para organizar datos, la implementación de punteros para enlazar nodos, la recursión como herramienta natural para recorrer estructuras jerárquicas, y el manejo de memoria dinámica con new. Además, el uso de GitHub como plataforma de control de versiones es una habilidad profesional esencial en el desarrollo de software moderno.